

Formik and Form Validation (CodeGrade)

 <https://github.com/learn-co-curriculum/python-formik-validation>  <https://github.com/learn-co-curriculum/python-formik-validation/issues/new>

Learning Goals

- Learn how to create and validate forms with Formik and Yup.

Key Vocab

- **Validation:** Validation is an automatic check to ensure that data entered is sensible and feasible.
- **Form:** An HTML form is used to collect user input. The user input is most often sent to a server for processing.

Introduction

Working with Form data on the front end can be tricky. We need to use the correct form html elements, write logic to validate data, and handle submissions. Formik is a React library that makes working with forms easy for us.

Setup

A full working solution is provided in the client directory. To start the front end server run `npm install` and `npm start` in the client directory. The site will be running on `http://localhost:4000/`

To run the server that stores the information the form creates enter the server/ directory and run:

- `pipenv install && pipenv shell` to enter your virtual environment.
- `export FLASK_APP=app.py` and `export FLASK_RUN_PORT=5555` to configure your Flask environment.
- `flask db upgrade` to generate your database.
- `python seed.py` to populate it.
- `python app.py` to run your development server.

The database on the server side has a unique constraint on the email. We do not need to worry about what is going on in the server code all we need to know is that the route for making calls to the api is called `/customers`.

Sign up form

In this lesson we will look at the code for a sign up form using React and Formik. The form will include input fields for email, name and age. Under the form there is a table that reflects what is stored in the database.

Lets look at the `Signup.js` file which contains the sign up form component.

The Formik library provides us a hook to give initial values to the form and write a `onSubmit` callback function to do something with the values that were submitted. We can use the `onSubmit` method to submit send the values to the server and store the values in the database. To do validation we can define a `validationSchema` which we will look at later on in the lesson.

```
const formik = useFormik({
  initialValues: {
    name: "",
    email: "",
    age: "",
  },
  validationSchema: formSchema,
  onSubmit: (values) => {
    fetch("customers", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
```

```

    body: JSON.stringify(values, null, 2),
  }).then(
    (res) => {
      if (res.status === 200){
        setRefreshPage(!refreshPage)
      }
    }
  )
},
});

```

Lets look at the JSX we need to write for the form. The form looks similar to a form we may create without any libraries. The useFormik hook gives us many useful functions to pass into the HTML attributes. Note how we do not have to write our own handleChange functions or logic to handle validation errors. The useFormik hook will do much of that work for us. We can get values using `formik.values` and errors using `formik.errors` .

```

return (
  <div>
    <h1>Customer sign up form</h1>
    <form onSubmit={formik.handleSubmit} style={{ margin: "30px" }}>
      <label htmlFor="email">Email Address</label>
      <br />
      <input
        id="email"
        name="email"
        onChange={formik.handleChange}
        value={formik.values.email}
      />
      <p style={{ color: "red" }}> {formik.errors.email}</p>
      <label htmlFor="name">Name</label>

```

```

    <br />

    <input
      id="name"
      name="name"
      onChange={formik.handleChange}
      value={formik.values.name}
    />
    <p style={{ color: "red" }}> {formik.errors.name}</p>

    <label htmlFor="age">age</label>
    <br />

    <input
      id="age"
      name="age"
      onChange={formik.handleChange}
      value={formik.values.age}
    />
    <p style={{ color: "red" }}> {formik.errors.age}</p>
    <button type="submit">Submit</button>
  </form>
  <table style={{ padding: "15px" }}>
    <tbody>
      <tr>
        <th>name</th>
        <th>email</th>
        <th>age</th>
      </tr>
      {customers === "undefined" ? (
        <p>Loading</p>
      ) : (
        customers.map((customer, i) => (

```

```
<>
  <tr key={i}>
    <td>{customer.name}</td>
    <td>{customer.email}</td>
    <td>{customer.age}</td>
  </tr>
</>
))
}}
</tbody>
</table>
</div>
);
```

Validation

For validation we will use the Yup library that meshes well with Formik. Yup is a schema builder for runtime value parsing and validation.

Lets look at the validation schema. Yup allows us to instantiate an object and define its shape using the shape function which takes in an object of the fields we want to validate. We can chain validation rules to create complex and strict validation rules that allow us to control input and create more robust applications.

```
const formSchema = yup.object().shape({
  email: yup.string().email("Invalid email"),
  name: yup.string().required("Must enter a name").max(15),
  age: yup.number().positive().integer().required("Must enter age").typeError('Please enter an Integer').max(125),
})
```

Lets look at the age schema as an example. We want to accept a `number()` which is `positive()` and is an `integer()` . We also want to make sure it is `required()` . Yup makes error messages easy to define. We can pass the error message we want to display in the broken rule by passing a string into the rule definition like we did here `required("Must enter age")` .

We can access these errors in the JSX by using the formik hooks errors attribute `<p style={{color:'red'}}> {formik.errors.name}</p>` . In this example we show the validation errors as a red `<p>` tag.

Now imagine how difficult it would be to write all the logic needed for the validation and error handling if we needed to do with without formik.

Conclusion

In this lesson we covered the basic functionality of creating a form and validating some inputs. With Formik we can create a more robust application and make sure we don't submit data to an API that may be unexpected or incorrect. This lesson was meant to show how Formik works at a fundamental level. The code we wrote is very explicit about exactly what Formik is doing. If you want to reduce the boilerplate code even more you can use front end components that wire the forms for us. Learn more about the built in Formik form components here [less verbose forms](https://formik.org/docs/overview#reducing-boilerplate) [⇒_\(https://formik.org/docs/overview#reducing-boilerplate\)](https://formik.org/docs/overview#reducing-boilerplate).

Solution code

```
import React, { useEffect, useState } from "react";
import { useFormik } from "formik";
import * as yup from "yup";
export const SignupForm = () => {
  const [customers, setCustomers] = useState([{}]);
  const [refreshPage, setRefreshPage] = useState(false);
  // Pass the useFormik() hook initial form values and a submit function that will
  // be called when the form is submitted

  useEffect(() => {
```

```
    console.log("FETCH! ");
    fetch("/customers")
      .then((res) => res.json())
      .then((data) => {
        setCustomers(data);
        console.log(data);
      });
  }, [refreshPage]);

const formSchema = yup.object().shape({
  email: yup.string().email("Invalid email").required("Must enter email"),
  name: yup.string().required("Must enter a name").max(15),
  age: yup
    .number()
    .positive()
    .integer()
    .required("Must enter age")
    .typeError("Please enter an Integer")
    .max(125),
});

const formik = useFormik({
  initialValues: {
    name: "",
    email: "",
    age: "",
  },
  validationSchema: formSchema,
  onSubmit: (values) => {
    fetch("customers", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
```

```
    },
    body: JSON.stringify(values, null, 2),
  }).then((res) => {
    if (res.status === 200) {
      setRefreshPage(!refreshPage);
    }
  });
},
});

return (
  <div>
    <form onSubmit={formik.handleSubmit} style={{ margin: "30px" }}>
      <label htmlFor="email">Email Address</label>
      <br />
      <input
        id="email"
        name="email"
        onChange={formik.handleChange}
        value={formik.values.email}
      />
      <p style={{ color: "red" }}> {formik.errors.email}</p>
      <label htmlFor="name">Name</label>
      <br />
      <input
        id="name"
        name="name"
        onChange={formik.handleChange}
        value={formik.values.name}
      />
      <p style={{ color: "red" }}> {formik.errors.name}</p>
    </form>
  </div>
);
```

```

<label htmlFor="age">age</label>
<br />

<input
  id="age"
  name="age"
  onChange={formik.handleChange}
  value={formik.values.age}
/>
<p style={{ color: "red" }}> {formik.errors.age}</p>
<button type="submit">Submit</button>
</form>
<table style={{ padding: "15px" }}>
  <tbody>
    <tr>
      <th>name</th>
      <th>email</th>
      <th>age</th>
    </tr>
    {customers === "undefined" ? (
      <p>Loading</p>
    ) : (
      customers.map((customer, i) => (
        <>
          <tr key={i}>
            <td>{customer.name}</td>
            <td>{customer.email}</td>
            <td>{customer.age}</td>
          </tr>
        </>
      ))
    )}
  </tbody>

```

```
    </table>  
  </div>  
);  
};
```

Resources

- [Formik](https://formik.org/docs/tutorial)  [_ \(https://formik.org/docs/tutorial\)](https://formik.org/docs/tutorial)
- [Formik Validation](https://formik.org/docs/guides/validation)  [_ \(https://formik.org/docs/guides/validation\)](https://formik.org/docs/guides/validation)
- [React](https://reactjs.org/docs/hello-world.html)  [_ \(https://reactjs.org/docs/hello-world.html\)](https://reactjs.org/docs/hello-world.html)

This tool needs to be loaded in a new browser window

Load Formik and Form Validation (CodeGrade) in a new window